VNUHCM - UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

PA5 Report

Group 6: PDF

CSC13002 - Introduction to Software Engineering

Author:
Nguyễn Tuấn An - 24127011
Nguyễn Hải Nam - 24127088
Nguyễn Hải Trọng - 24127311
Phạm Đinh Gia Khánh - 24127420
Võ An Phú - 24127494

Lecturer:
Ms. Nguyễn Thị Minh Tuyền
Mr. Hồ Tuấn Thanh
Mr. Mai Anh Tuấn

July 21, 2026

Table of Contents

- A - Test Plan and Test Cases
  - 1. Test Plan
    - 1.1 Test Objectives and Scope
      - 1.1.1 Test Objectives
      - 1.1.2 Test Scope
        - 1.1.2.1 In-Scope
        - 1.1.2.2 Out-of-Scope
    - 1.2 Features to be tested
    - 1.3 Test Environment and Tools
      - 1.3.1 Operating and Execution Environment
      - 1.3.2 Test and Inspection Tools
    - 1.4 Test Schedule and Responsibilities
      - 1.4.1 Test Responsibilities
      - 1.4.2 Test Schedule
    - 1.5 Entry and Exit Criteria
      - 1.5.1 Entry Criteria
      - 1.5.2 Exit Criteria
  - 2. Test Cases
  - 3. Test Execution
  - 4. Bug Report
    - 4.1 Bug Report File
    - 4.2 Test Summary
      - 1. Test Objectives
      - 2. Test Execution Summary
- C - Reflective Report
  - 1. Team experience
    - What went well:
    - Challenges during the project:
  - 2. Spec Kit experience
    - How was the experience of using Spec Kit for specification-driven development?
    - Benefits compared to traditional development:
    - Limitations compared to traditional development:
  - 3. AI tools usage
    - How AI coding tools contributed to the project:
    - Effective aspects:
    - Limitations encountered:

- 4. SDLC feedback
- 5. Individual contributions
  - Nguyễn Tuấn An (24127011) — Project Manager & DevOps, Full-Stack Developer
  - Nguyễn Hải Nam (24127088) — QA Engineer / Full-Stack Developer
  - Nguyễn Hải Trọng (24127311) — Backend & Database Lead / Full-Stack Developer
  - Phạm Đinh Gia Khánh (24127420) — UI/UX Designer & Full-Stack Developer
  - Võ An Phú (24127494) — Frontend Lead & Full-Stack Developer

A - Test Plan and Test Cases

Work performed by: Nguyễn Hải Nam | Reviewed and Edited by: Võ An Phú

1. Test Plan

1.1 Test Objectives and Scope

1.1.1 Test Objectives

- **Primary Functional Correctness:** To verify that the core user flows of the FOZ forum perform according to specifications without critical errors or UI breakdowns.
- **Role-Based Access Control:** To ensure proper privilege segregation across user roles (`Guest` , `Standard User` , `Moderator` , and `Administrator`), specifically validating that elevated administrative actions are restricted strictly to authorized admin accounts.
- **AI Feature Validation:** To validate the functional correctness of the AI-powered `Post Summarization` feature, ensuring that it produces accurate TL;DR summaries via the Google Gemini API integration and handles missing API keys smoothly via fallback mechanisms.
- **Defect Tracking & Reporting:** To execute manual test cases, identify bugs or regressions, and log detailed reports for developers to fix.

1.1.2 Test Scope

1.1.2.1 In-Scope

- **Target Use Cases:** Manual functional testing of the 6 chosen use cases: UC-07, UC-08, UC-10, UC-12, UC-14, UC-18.
- **Full-Stack Integration:** Web frontend interaction (Vite/Client) interacting with backend API routes (`/api/v1/...`) and PostgreSQL persistence via Drizzle ORM.
- **Browser Compatibility:** Basic UI and functional checks on modern browsers (e.g., Google Chrome, Microsoft Edge).

1.1.2.2 Out-of-Scope

- **Non-functional Requirements:** Load testing, stress testing, high-concurrency scaling, and deep security penetration auditing.
- **Unselected Use Cases:** Features outside the 6 selected use cases (such as authorization or avatar customization).
- **Native Mobile Applications:** Testing is strictly limited to desktop/responsive web environments.
- **Test Automation:** Writing or executing automated test scripts (e.g., Unit, E2E testing).

1.2 Features to be tested

- UC-07 - Browse Feed (Feed Generation)
- UC-08 - Search Content (Search & Discovery)
- UC-10 - Create Post (Post Creation)
- UC-12 - Leave Comments (User Interaction)
- UC-14 - Request AI Summary (Post Summarization)
- UC-18 - Forward Report to Admin (Administration & Moderation)

1.3 Test Environment and Tools

1.3.1 Operating and Execution Environment

- **Operating System:** Windows 11 / Linux (Ubuntu 24.04 LTS / WSL2)
- **Frontend Environment:** Vite, React, dev server on `http://localhost:5173`.
- **Backend Environment:** Express, Drizzle ORM, dev server on `http://localhost:4000`.
- **Database Container:** PostgreSQL `v16+` running via Docker exposed on port `5432`.

1.3.2 Test and Inspection Tools

- **Web Browsers:** Google Chrome and Microsoft Edge for manual functional UI execution and responsive viewport checks.
- **Browser Developer Tools:** Chrome DevTools and Microsoft Edge DevTools (Console, Network, etc.)
- **Database Inspection GUI:** Drizzle Studio (`npx drizzle-kit studio`) to inspect real-time table state changes in PostgreSQL (users, posts, comments, roles, etc.).
- **External Services & API Keys:** Google AI Studio API key for AI summarization testing.
- **Source Code & Environment Management:** Git version control, Docker Engine for database containerization, VS Code and Antigravity IDE.

1.4 Test Schedule and Responsibilities

1.4.1 Test Responsibilities

Role / Responsibility	Assigned Team Member	Specific Tasks & Deliverables
Test Planning	Nguyen Hai Nam	Write specific Test Plan for team.
Test Environment and Data Preparation	Nguyen Hai Nam & Nguyen Tuan An	Set up the local testing environment (DB container, backend, frontend). Prepare test data including mock user accounts, pre-existing posts, and valid/invalid Gemini API keys for AI testing.

Role / Responsibility	Assigned Team Member	Specific Tasks & Deliverables
Test Case Design (UC-07 & UC-08)	Nguyen Tuan An	Refine and write test cases for **UC-07 (Browse Feed)** and **UC-08 (Search Content)**.
Test Case Design (UC-10)	Nguyen Hai Nam	Refine and write test cases for **UC-10 (Create Post)**.
Test Case Design (UC-12)	Vo An Phu	Refine and write test cases for **UC-12 (Leave Comment)**.
Test Case Design (UC-14)	Nguyen Hai Trong	Refine and write test cases for **UC-14 (Request AI Summary)**.
Test Case Design (UC-18)	Pham Dinh Gia Khanh	Refine and write test cases for **UC-18 (Forward Report to Admin)**.
Test Case Review & Quality Assurance (UC-07 & UC-08)	Nguyen Hai Nam & Pham Dinh Gia Khanh	Review and refine all test cases for **UC-07 & UC-08**
Test Case Review & Quality Assurance (UC-10 & UC-12)	Nguyen Tuan An & Nguyen Hai Trong	Review and refine all test cases for **UC-10 & UC-12**
Test Case Review & Quality Assurance (UC-14 & UC-18)	Pham Dinh Gia Khanh & Vo An Phu	Review and refine all test cases for **UC-14 & UC-18**
Test Case Review & Quality Assurance for AI-generated test cases	All Team Members	Review and refine all test cases for AI-generated test cases
Test Execution and Bug Logging	All Team Members	- Execute assigned manual test cases using Cross-testing approach, accurately recording the **Test case ID**, **execution date**, **status** (Pass/Fail), and **actual results**. - Log all discovered defects into the Bug Report, ensuring every failed test case is strictly linked to at least one **bug ID**. - Record bug details, including **bug description**, **steps to reproduce**, **expected vs. actual results**, **severity**, and **status**.
Retesting and Final Run Verification	Nguyen Hai Nam & Vo An Phu	Perform Retesting on fixed bugs and execute the Final

Role / Responsibility	Assigned Team Member	Specific Tasks & Deliverables
		Run to ensure all previously failed test cases are successfully passed and bug statuses are updated to 'Closed/Fixed'.
Test Report and Document Finalization	Nguyen Hai Nam & Nguyen Hai Trong	- Document Bug Report and Test Summary. - Compile, format, and finalize the complete Software Testing Document.

1.4.2 Test Schedule

Phase	Milestone / Task Category	Timeframe	Primary Assignees
Phase 1	**Test Planning & Preparation:** Finalize the test plan, set up the local testing environment, and prepare all required test data (mock accounts, pre-existing posts, API keys).	Aug 11 - Aug 15, 2026	Nguyen Hai Nam & Nguyen Tuan An
Phase 2	**Test Case Design & Review:** Refine and design test cases across 6 selected use cases. Review and refine all AI-generated test cases. Conduct peer reviews to ensure optimal test coverage and validity.	Aug 16, 2026	All Team Members
Phase 3	**First Run Execution & Bug Logging:** Perform cross-testing on the application. Accurately record execution status	Aug 17 - Aug 23, 2026	All Team Members

Phase	Milestone / Task Category	Timeframe	Primary Assignees
	(Pass/Fail) and systematically log all discovered defects.		
Phase 4	**Bug Fixing & Retesting (Final Run):** Allocate time for defect resolution. Perform retesting on previously failed test cases to verify fixes and achieve a 100% pass rate.	Aug 24, 2026	Nguyen Hai Nam & Vo An Phu (Retesting)
Phase 5	**Reporting & Finalization:** Compile Test Summary metrics, verify bug ID traceability, format and finalize the document.	Aug 25, 2026	Nguyen Hai Nam & Nguyen Hai Trong

1.5 Entry and Exit Criteria

1.5.1 Entry Criteria

- **Feature Implementation & Code Stability:** Code for all 6 selected use cases (UC-07, UC-08, UC-10, UC-12, UC-14, UC-18) is fully integrated and ready for testing.
- **Environment & Server Readiness:**
  - The PostgreSQL database Docker container is running on port `5432` with all database migrations applied.
  - The backend server (`http://localhost:4000`) runs without errors.
  - The frontend server (`http://localhost:5173`) runs without errors .
- **Test Data Seeded:** Mock user accounts (for all roles) and sample data (posts, comments, etc.) are loaded into the database.
- **Test Cases Approved:** All functional test cases have been reviewed, refined, and approved by the team.
- **API Keys Configured:** `.env` variables are set, including both valid and invalid Gemini API keys to verify the AI fallback mechanism.

1.5.2 Exit Criteria

- **100% Execution:** All designed test cases have been executed.

- **Results Logged:** Execution dates, status (Pass/Fail), and actual results are fully documented for every test case.
- **Defect Traceability:** Every test case that failed during the First Run is linked to a specific Bug ID.
- **Bugs Fixed & Retested:** High-severity/critical defects have been resolved by developers. Retesting (Final Run) confirms these tests now pass.
- **Report Finalized:** The Test Summary metrics and Bug Report are fully populated and formatted for submission.

2. Test Cases

> The below link contains the manual functional test cases designed by our team.

Team Manual Functional Test Cases `./Individual-Test-Execution/Individual-Manual-Test-Cases-Execution.xlsx`

> The link below contains our team's document on reviewing, refining, and adding AI-generated test cases.

Team's Document on Reviewing, Refining, and Adding AI-generated Test Cases `./Review-AI-Test-Case/`

3. Test Execution

> The below link contains the manual functional test execution with required records performed by our team.

Team Manual Functional Test Execution `./Individual-Test-Execution/Individual-Manual-Test-Cases-Execution.xlsx`

4. Bug Report

4.1 Bug Report File

> The link below contains the bug report performed by our team.

Team Bug Report `./Individual-Bug-Report/Individual-Bug-Report.xlsx`

4.2 Test Summary

1. Test Objectives

The objective of this test cycle was to validate the functionality of key features in the FOZ application, specifically focusing on:

- **UC-07 (Browse Feed):** Verifying the feed functionality across all user roles, sorting, ranking algorithms, scroll behaviors, and offline states.

- **UC-08 (Search Content):** Validating the search capability across posts and profiles, including hashtag filtering, query limits, empty states, and offline conditions.
- **UC-10 (Create Post):** Verifying that users can successfully create posts with text and media, including error handling for boundary limits and network issues.
- **UC-12 (Leave Comment):** Testing comment submissions across roles, verifying nested replies, UI responsivness, XSS security, character truncation, and duplicate request blocking.
- **UC-14 (Request AI Summary):** Validating the generation of AI summaries for posts and comments, including cache behavior, external API resiliency, and access control.
- **UC-18 (Escalate Report):** Validating the ability for moderators to escalate reports of serious or complex violations to administrators, including handling concurrent edge cases, deleted content, and input validation.

2. Test Execution Summary

Feature / Use Case	Tester	Test Date	Total Cases	Initial Failed (Defects Logged)	Final Failed	Final Passed	Final Pass Rate
UC-07 (Browse Feed)	Nguyễn Tuấn An	2026-08-21	11	3	0	11	100%
UC-08 (Search Content)	Nguyễn Tuấn An	2026-08-22	15	4	0	15	100%
UC-10 (Create Post)	Nguyễn Hải Nam	2026-08-17	17	1	0	17	100%
UC-12 (Leave Comment)	Võ An Phú	2026-08-17	24	5	0	24	100%
UC-14 (Request AI Summary)	Nguyễn Hải Trọng	2026-08-23	17	5	0	17	100%
UC-18 (Escalate Report)	Phạm Đinh Gia Khánh	2026-08-22	19	7	0	19	100%
Total			**103**	**25**	**0**	**103**	**100%**

(Note: During the initial testing phase, 25 test steps had associated Defect IDs logged. All 25 failed/defective test steps have since been addressed, re-tested, and successfully verified. The application now yields a clean 100% pass rate across these core use cases.)

C - Reflective Report

Work performed by: Group 06 | Reviewed by: Nguyễn Tuấn An and Võ An Phú | Edited by: Nguyễn Tuấn An

1. Team experience

What went well:

- Agile Scrum collaboration: the team successfully adopted the Scrum methodology across all 5 PAs. We maintained regular sprint planning sessions, daily scrums, sprint reviews, and tracked progress using Jira.
- Clear domain leadership with fullstack ownership: While every member operated as a full-stack engineer, assigning domain ownership (Project Management, QA/Testing, Backend/Database, UI/UX Design, and Frontend Architecture) enabled efficient guidance and specialized accountability.
- Strict quality control: mandatory peer reviews, automated TypeScript compile checks (`tsc --noEmit`), Zod trust-boundary schema validations, and extensive Vitest integration test suites ensured high code quality and zero regressions across sprints.
- Architectural Cohesion: Establishing a single source of truth in `src/shared/` for Zod schemas, DTOs, and enums eliminated API contract discrepancies between the React frontend and Express backend.
- Successful Multi-Cloud Production Deployment: The team successfully delivered and deployed a production-grade web application with a live frontend on Vercel Edge CDN, containerized backend API on Railway, fully managed PostgreSQL on Aiven, image CDN on Cloudinary, and Google Cloud Platform integrations (OAuth 2.0 and Gemini API).
- Gradually improve our product based on lecturer's feedback from each PA.
- Successfully apply software engineering foundations on a real project (e.g. testing, diagram drawing, etc.)

Challenges during the project:

- Time management & heavy workload: balancing heavy course commitments, midterm exams, and intensive development sprints required diligent backlog prioritization and front-loading critical tasks early in each sprint.
- AI results not good on document tasks: AI shows flaws when dealing with document writing, it lacks creativity and logical coherence.
- Deploy challenge: as we do not have enough facility for solid deployment services, we only choose free-tier with limited bandwidth, storage, and computing power. It is not

stable for production-grade deployment and we have changed 2 services for backend and database hosting services due to resource limitation.
- Lack of hands-on experience, especially when dealing with web development.

2. Spec Kit experience

How was the experience of using Spec Kit for specification-driven development?

Using Spec Kit transformed our development approach from informal prompting into a structured, specification-driven engineering lifecycle (`specify` -> (`clarify`) -> `plan` -> `tasks` -> (`analyze`) -> `implement`). Defining formal feature specifications, data models, and API contracts before writing code forced the team to thoroughly evaluate edge cases, validation boundaries, and dependencies upfront. This systematic workflow minimized rework and ensured that the implementation directly fulfilled the functional requirements.

Benefits compared to traditional development:

- Early disambiguation & reduced rework: Spec Kit surfaces ambiguous requirements during the `specify` and `clarify` phases, resolving architectural questions before they manifest as costly bugs in production. The `analyze` phase further cross-checks the spec, plan, and tasks for gaps and inconsistencies before implementation begins, catching coverage issues that manual reviews would otherwise miss.
- Granular, dependency-ordered task decomposition: The `tasks.md` breakdown allowed our team members can review before actual implementation.
- Living architectural documentation: The artifacts generated in the `specs/` directory serve as clear, evergreen documentation of the system's design, API contracts, and testing requirements, making onboarding and code reviews seamless.
- Enhanced AI context & code generation quality: Spec Kit provides AI coding assistants with rich contextual anchors (schemas, project constitution, directory conventions), resulting in higher accuracy and minimal hallucinations.

Limitations compared to traditional development:

- Upfront process overhead: Generating specs, research notes, contracts, and task checklists requires significant initial time investment, which can feel heavy for trivial UI adjustments or minor bug fixes.
- Tooling and prompt dependency: Effective use of Spec Kit requires familiarity with specific prompt commands and token management; poor prompts can lead to oversized task lists or generic boilerplate.
- Token limitation: Spec Kit or any vibe engineering methods often depend on AI tokens. The limitation of tokens and decent models can sometimes make it difficult to generate a quality codes, specifications.

- Setup Spec Kit and skill sets is time-consuming and requires steep learning curve. Different models, OS, IDEs, code editors, etc. require different set up (e.g. setup Spec Kit on VS Code is different from Antigravity)

3. AI tools usage

How AI coding tools contributed to the project:

AI coding assistants such as Google Antigravity (Gemini 3.7 Flash, Gemini 3.1 Pro), GitHub Copilot (GPT-4o, GPT-5.4 mini), etc. deeply integrated into every phase of the software development lifecycle. They assisted in:

- Rapidly scaffolding backend route handlers, Drizzle schemas, and Zod validation DTOs.
- Designing responsive React UI components with Tailwind CSS based on design specifications.
- Generating comprehensive Vitest integration tests covering edge cases, authentication gating, and interaction flows.
- Executing SpecKit specification and planning workflows from natural-language requirements.
- Rapidly write draft version for the document required for PA submissions.
- Maintaining continuous development transparency via automated logging in `ai-logs/<username>.md`.
- Designing UI for app (Google Stitch)
- Giving information about exploring and choosing appropriate tech stack, cloud services for the project.

Effective aspects:

- Accelerated Boilerplate & CRUD Development: Scaffolding REST endpoints, data access layers, and type definitions took minutes instead of hours, significantly boosting team velocity.
- Comprehensive Test Case Generation: AI tools excelled at identifying boundary conditions (e.g., empty arrays, invalid JWTs, SQL injection payloads, long unicode inputs, rate limits) and writing extensive automated test suites.
- Strict Architectural Adherence: When supplied with our project constitution (`constitution.md`) and routing guidelines (`AGENTS.md`), the AI consistently enforced team conventions (e.g., `/api/v1` route prefixing, modular client-server isolation).
- Rapid Debugging & Error Diagnosis: AI assistants quickly identified root causes for subtle bugs, such as session cookie ID mismatches and Vite proxy misconfigurations.

Limitations encountered:

- Superficial "Smoke Test" Generation: AI models sometimes generated shallow assertions (e.g., verifying `typeof fn === 'function'` or duplicating sorting logic in tests) rather

than writing genuine end-to-end database queries, requiring manual test refinement during QA reviews.
- Context Degradation in Long Conversations: Over prolonged sessions, AI agents occasionally lost track of existing utility functions or attempted to re-implement already available helpers.
- Outdated Suggestions: AI occasionally suggested not suitable services. When our team first asked AI about services for deployment, the AI suggest those with low bandwidth leading to the service change later.
- Learning coding skills from AI: learning only basics from AI, not enough for advanced concepts and professional software development.

4. SDLC feedback

So far, we have reached the final PA and have a good understanding of the SDLC process and course structure. Overall, the course and project are well-organized and the team has learned a lot, strengthening both technical and soft skills. The team sincerely thanks the instructors for their guidance.

Below are some constructive suggestions for improving the Software Development Life Cycle (SDLC) process and course structure:

1. Detailed and Actionable Grading Feedback:
   - Feedback on previous PA submissions was occasionally brief or lacked specific breakdowns, making it difficult for the team to understand exact grading criteria or identify which specific sections lost marks. Also there should be more instruction or more meeting in class for both discussions and clarifications.
   - Suggestion: Provide a transparent, specific feedback comments for each milestone by pointing out exactly what students have done good or not good, . This will give student teams concrete guidance on areas needing improvement before the subsequent submission.
2. Up-to-date and Centralized Course Documentation:
   - Some requirement documents and template files contained outdated instructions or ambiguous specifications that required clarification across different communication channels (e.g. Moodle discussion forum).
   - Suggestion: Maintain a single, updated repository for all PA templates, grading criteria, and technical requirements. Clarify modern stack expectations (e.g., TypeScript, modern ORMs, and cloud deployments) early in the semester. Also should provide a template for reports for each PA to help students save time when defining the format.
3. The documentation tasks is overhead:
   - Our team thinks that the main value of Agile/Scrum is about development and minimal documentation, especially for small team size under short period of time and the huge workload of this course.

- Suggestion: Optimize the documentation so that it is both more condensed, and more realistic for small team size.

5. Individual contributions

Nguyễn Tuấn An (24127011) — Project Manager & DevOps, Full-Stack Developer

As Project Manager, I organized sprint planning sessions, coordinated task assignments on Jira, and also established the development workflow and multi-cloud deployment pipeline across Vercel, Railway, Aiven, and Cloudinary. I implemented the Google OAuth 2.0 social login flow, the Post Summarize (TL;DR) AI service with resilient fallback chains, and led the AI summarization test suite refinement, fixed a huge number of bugs, manage the document for submission. Through this project, I gained invaluable practical experience in orchestrating end-to-end cloud infrastructure, managing agile team dynamics, and enforcing specification-driven AI development with SpecKit.

Nguyễn Hải Nam (24127088) — QA Engineer / Full-Stack Developer

During this project, I worked on core features like the admin dashboard and user interactions, plus a few smaller additions like hashtag feature. I also handled the documentation for all PAs and fixed bugs across the codebase. I have learned a lot—from understanding the overall architecture and system flow, to learning GitHub Speckit and various frameworks and putting them into practice. It was an unforgettable experience for me.

Nguyễn Hải Trọng (24127311) — Backend & Database Lead / Full-Stack Developer

During the project, I designed the core database schema, authored the Vision Document, developed comprehensive Use-Case Specifications, and implemented key backend features like post creation. For PA5, I designed detailed manual test cases for the AI Summary feature and reviewed and refined AI-generated test cases for the Profile Management and Search & Discovery functional groups. I faced significant challenges, including occasionally missing deadlines and a laptop failure during PA2 that impacted our progress. I am deeply grateful to my teammates for stepping up to cover my tasks during that time. Ultimately, this experience built my foundational industry knowledge by teaching me the mechanics of TDD, the practical application of Spec Kit, and how to collaborate effectively with AI Agents and my team.

Phạm Đinh Gia Khánh (24127420) — UI/UX Designer & Full-Stack Developer

During this project, I built up some core features like searching, moderator,... as well as handled the documentation for PAs, fixed bugs. I have learnt a lot of things, especially about the architecture, the workflow and how to set up speckit. I think these experience would assist me a lot in my future projects. Besides, a big shoutout to my teammate, we've done such a great job working like a cohesive team.

Võ An Phú (24127494) — Frontend Lead & Full-Stack Developer

In this project, I think my contribution was a lot, from setting up the project speckit foundation, team AI usage, the web scaffold, key feature like AuthN, frontend UI overhaul to documentations. I have learnt a lot about architecture and data flow of the whole system throughtout this project. Overall it was pleasant